Institut Mines - Télécom -- Télécom SudParis.

Évry, France.

September 29th, 2017.

Author: Patricia Reinoso

WRAPPER FOR CAPE

TABLE OF CONTENTS

INTRODUCTION

OpenMP is a tool for multi-thread and shared-memory programming. It is composed by a group of directive that can be inserted into a sequential program to indicate parallel regions, divide blocks of code among threads, distribute loop iterations among threads, and thread synchronization. [2]

Checkpointing Aided Parallel Execution (CAPE) is a tool currently under development, that provides solutions to optimize code for distributed-memory machines, the same way OpenMP does for shared-memory machines.

A wrapper was created in order to automate the integration of the parallel code of CAPE into the sequential code of OpenMP statements. This task was fulfilled using Turing Extended Language (TXL). TXL allowed the definition of C-syntax grammars for OpenMP and for CAPE, and a set of rules to identify and transform OpenMP directives into CAPE statements.

TXL

Turing Extended Language (TXL) is a hybrid functional/rule-based programming language designed to support computer software analysis and source transformation tasks. It includes parametrization, recursion, deep pattern search, unification and implicit iteration, therefore it eases complex computing problems that involve structural analysis and transformation of formal notations, such as programming languages. In particular, this language fits perfectly for developing parsers, semantic analyzers, translators, interpreters, and extensions and dialects of existing programming languages. [1]

FILE STRUCTURE

The wrapper is composed by 3 types of files:

***.Grm :** grammar definition. Three grammars are considered: C grammar, OpemMP grammar and CAPE grammar. All the transformation rules were defined based on structures specified in these files.

***.Rul :** set of rules that executes the transformation from OpenMP statements to CAPE statements. The rules were divided in various files in order to make it more readable.

openmptocape.txl : is the main file that links all the other files and executes the corresponding transformations.

All the files are described more in detail below:

C.Grm

- Contains the basic grammar for C with GNU extensions.
- Grammar taken from the TXL Resources, available for public use. [5]

CAPE.Grm

- Contains the grammar definition for CAPE

CCommentOverrides.Grm

- Contains the grammar extensions for C that allows parsing comments in C files.
- Grammar taken from the TXL Resources, available for public use. [5]

CGnuOverrides.Grm

- Contains the grammar extensions for C to operate with GNU C files with some exceptions.
- Grammar taken from the TXL Resources, available for public use. [5]

OMP.Grm

- Contains the grammar definition for OpenMP.

omptocape.Txl

- Main file that executes the transformation of OpenMP statements to CAPE statements.
- All files containing the transformation rules and C grammar are included in this file.
- The set of rules and definitions in this files are presented in Table 1.

Rules
main
Functions
changeUnit CompilationUnit [declaration_or_function_definition]
checkOMPInclude
treatGlobalDeclaration
treatGlobalThreadPrivate
treatFunction

checkDirective
replaceEnvRoutInFunctions
addGlobalVarsToMain

Table 1. Set of rules in the file omptocape.Txl

clausesRules.Rul

- Contains the transformation rules for the OpenMP clauses to CAPE clause statements.
- The clauses considered are: default shared, default none, shared, private, firstprivate, lastprivate, reduction, copyin and copyprivate.
- The set of rules and definitions in this files are presented in Table 2.

New definitions
define clauses
Functions
replaceClauses
Default Clause Replacement
replaceDefaultClause
replaceDefaultSharedClause
replaceDefaultNoneClause
Shared Clause Replacement
replaceSharedClause
addSharedClause Variable [reference_id]
Private Clause Replacement
replacePrivateClause
addPrivateClause Variable [reference_id]
FirstPrivate Clause Replacement
replaceFirstPrivateClause
addFirstPrivateClause Variable [reference_id]

LastPrivate Clause Replacement
replaceLastPrivateClause
addLastPrivateClause Variable [reference_id]
Reduction Clause Replacement
replaceReductionClause
addReductionClause Operator [omp_operator] Variable [reference_id]
Copyin Clause Replacement
replaceCopyinClause
addCopyinClause Variable [reference_id]
CopyPrivate Clause Replacement
replaceCopyPrivateClause
addCopyPrivateClause Variable [reference_id]
Nowait Clause Replacement
replaceNowaitClause
Clauses in Parallel Construct Replacement
addParallelDefaultClauses Clause [parallel_construct_clauses]
addParallelSharedClauses Clause [parallel_construct_clauses]
addParallelPrivateClauses Clause [parallel_construct_clauses]
addParallelFirstPrivateClauses Clause [parallel_construct_clauses]
addParallelFirstPrivateClauses Clause [parallel_construct_clauses]
addParallelLastPrivateClauses Clause [parallel_construct_clauses]
addParallelReductionClauses Clause [parallel_construct_clauses]
addParallelCopyinClauses Clause [parallel_construct_clauses]
Clauses in For Construct Replacement
addForPrivateClauses Clause [for_construct_clauses]
addForFirstClauses Clause [for_construct_clauses]
addForLastPrivateClauses Clause [for_construct_clauses]

addForReductionClauses Clause [for_construct_clauses]
addForNowaitClauses Clause [for_construct_clauses]
Clauses in Sections Construct Replacement
addSectionsPrivateClauses Clause [sections_construct_clauses]
addSectionsFirstPrivateClauses Clause [sections_construct_clauses]
addSectionsLastPrivateClauses Clause [sections_construct_clauses]
addSectionsReductionClauses Clause [sections_construct_clauses]
addSectionsNowaitClauses Clause [sections_construct_clauses]
Clauses in Single Construct Replacement
addSinglePrivateClauses Clause [single_construct_clauses]
addSingleFirstPrivateClauses Clause [single_construct_clauses]
addSingleCopyPrivateClauses Clause [single_construct_clauses]
addSingleNowaitClauses Clause [single_construct_clauses]

Table 2. Set of rules in the file clausesRules.Rul

constructRules.Rul

- Contains the transformation rules for the OpenMP constructs to CAPE statements.
- The constructs considered are: threadprivate, parallel, for, sections, single, master,critical, barrier, automatic.
- The set of rules and definitions in this files are presented in Table 3.

Definitions
directives
Redefinitions
preprocessor
Rules
replaceConstruct
Functions
Thread Private Construct Replacement

replaceThreadPrivate
addVarThreadPrivate Variable [decl_specifiers]
Parallel Construct Replacement
replaceParallel
replaceReductionBooleanInParallel Clauses [repeat parallel_construct_clauses]
checkReductionClauseInParallel
Parallel For Construct Replacement
replaceParallelFor
Parallel Sections Construct Replacement
replaceParallelSections
For Construct Replacement
replaceFor
replaceForNowait
replaceRegularFor
replaceReductionBooleanInFor
checkReductionClauseInFor
Sections Construct Replacement
replaceSections
replaceRegularSections
replaceSectionsNowait
addSection
replaceReductionBooleanInSections
checkReductionClauseInSections
Single Construct Replacement
replaceSingle
Master Construct Replacement
replaceMaster

Critical Construct Replacement
replaceCritical
Barrier Construct Replacement
replaceBarrier
Automic Construct Replacement
replaceAutomic
Flush Construct Replacement
replaceFlush

Table 3. Set of rules in the file constructRules.Rul

datatypeRules.Rul

- Contains the transformation rules for C data type to CAPE data type.
- The C data types identified are: char, double, float, int, long, unsigned char, unsigned float and unsigned int.
- The set of rules and definitions in this files are presented in Table 4.

Definitions
data_type
Functions
replaceVarType
replaceCharType
replaceDoubleType
replaceFloatType
replaceIntType
replaceLongType
replaceUnsignedCharType
replaceUnsignedFloatType
replaceUnsignedIntType

Table 4. Set of rules in the file datatypeRules.Rul

environmentRoutRules.Rul

- Contains the transformation rules for the OpenMP execution environment rules to CAPE execution environment rules.
- The execution environment rules considered are: set number of threads, get number of threads and get thread number.
- The set of rules and definitions in this files are presented in Table 5.

Definitions
environment_routine
Redefinitions
reference
compound_statement_body
Rules
replaceEnvRoutine
Functions
replaceEnvRoutSetNum
replaceEnvRoutGetNum
replaceEnvRoutGetThread

Table 5. Set of rules in the file environmentRoutRules.Rul

operatorRules.Rul

- Contains the transformation rules for OpenMP operators included in the reduction clause.
- The operators considered are: +, *, max, min.
- The set of rules and definitions in this files are presented in Table 6.

Definitions
operators
Functions
replaceOperator
replaceSumOperator
replaceMulOperator
replaceMaxOperator
replaceMinOperator

Table 6. Set of rules in the file operatorsRules.Rul

varDeclarationRules.Rul

- Contains the transformation rules for C variable declaration to CAPE variable declaration.
- The following cases are considered: single variable declarations, 1-dimension arrays, 2-dimension arrays, list of all the previous.
- The set of rules and definitions in this files are presented in Table 7.

Redefinitions
declaration
Rules
replaceDeclaration
replaceVariableDeclaratio
replaceArrayDeclaration
replace2DimArrayDeclaration
replaceDeclarationList
Functions
addVarDeclaration VarType [decl_specifiers] Variable [init_declarator]

Table 7. Set of rules in the file varDeclarationRules.Rul

HOW TO EXECUTE

To execute the wrapper it is necessary to install TXL and execute the following command line.

$ txl omptocape.txl *CProgramToTransform.c*

All the files together

IN PROGRESS

The Wrapper created is not finished and is not completely robust. The following aspects are details that were not finished or that can be improved.

- The repeat of clause after the omp construct were not recognized as wished by the compiler, for unknown reasons. Too much time was spent trying to solve this problem, so at the end it was decided to provide a simple solution even if it was not the best one. The solution consist in: for each construct that may have several clauses, there is a replacement function for each possible clause, that deconstruct the clause and the replace it according to the clause type.
- On the list of arguments on the definitions of each [omp_clauses] (corresponding to variable names), it is not guarantee that it possess at least one element.
- The program does not verify duplicated include statements when the needed ones are added to the C program.
- The parameters in the for statements of the [cape_for], [cape_parallel_for] and [cape_automic_critical] construct definitions may be more general.
- The list of cape arguments inside [cape_begin] and [cape_end] may be more specific, restricting the argument types according to the construct.
- To be able to transform from OpenMP to CAPE it is necessary to create a more general type composed as the union of both type of statements. For example:

  Operators are defined as following:

```
define operators
                [omp_operator]
        |       [cape_operator]
```

```
    end define
```

In order for the program to work, it was necessary to include [operators] instead of [cape_operator] as non-terminal symbol in the reduction clauses definition for CAPE. In this way, some inconsistencies may be inserted in the program if no attention is payed.

The same happens to [clauses] in the CAPE definition of the constructs and [data_type] in variable declarations.

RECOMMENDATIONS

- JTxlDb is a debugger that provides a graphical interface to debug TXL programs. It was very useful during the development of the wrapper [4].
- Follow the style recommendations for TXL programs in order to assure readability and ease maintenance [3].

REFERENCES

[1] About TXL. Retrieved September 05, 2017, from http://www.txl.ca/nresources.html

[2] Barney, B. (n.d.). Retrieved September 05, 2017, from https://computing.llnl.gov/tutorials/openMP/

[3] Coding Conventions for TXL. Retrieved September 29, 2017, from http://www.txl.ca/docs/TXLstyle.pdf

[4] JTxlDb: A Java/Sing wrapper for the TXL debugger. Retrieved September 29, 2017, from http://selab.fbk.eu/tiella/jtxldb/

[5] TXL Resources. Retrieved September 05, 2017, from http://www.txl.ca/nresources.html